UNIVERSITY MOBILE APPLICATION DEVELOPMENT LABORATORY

Practical Manual — Lab 01

Lab Setup: Installing Android Studio and the Android Emulator, Exploring the IDE, and Building Your First “Hello Mobile World” App in Kotlin

---

Target Audience

Undergraduate students with little or no prior mobile development experience

Reference Environment

Operating System: Windows 10 / 11 (64-bit)
Android Studio — latest stable release
Android SDK — installed via Android Studio
Programming Language: Kotlin
Android Emulator (AVD) — required
Physical Android device — optional

Estimated Duration: 2 hours

Part 1 — Introduction

1.1 What is Android Studio?

Android Studio is the official Integrated Development Environment (IDE) for Android app development, published by Google and built on JetBrains' IntelliJ IDEA platform. It bundles everything needed to write, build, debug, and test Android applications in one application: a code editor with intelligent code completion, a visual layout designer, a build system (Gradle), device emulation tools, and performance profiling tools.

1.2 What is the Android SDK?

The Android Software Development Kit (SDK) is the collection of libraries, platform tools, and APIs that let a computer compile, package, and run Android applications. It includes the Android platform itself (one version per Android release), command-line tools such as adb (Android Debug Bridge), and the Android Emulator. Android Studio installs and manages the SDK automatically, but it is technically a separate component from the IDE — you could, in principle, build Android apps from only the SDK and a text editor, though Android Studio makes this dramatically easier.

1.3 What is Kotlin, and Why Use It for Android?

Kotlin is a modern, statically typed programming language developed by JetBrains that runs on the Java Virtual Machine and compiles to the same bytecode used by Java-based Android apps. Since 2019, Google has recommended Kotlin as the preferred language for new Android development. Kotlin offers several practical advantages for this course:

- Conciseness: many common Android patterns require noticeably less boilerplate code than the equivalent Java.
- Null safety: Kotlin's type system distinguishes nullable from non-nullable types at compile time, preventing a large class of NullPointerException crashes before the app ever runs.
- Full interoperability with Java: Kotlin can call existing Java-based Android APIs directly, so it works seamlessly with the rest of the Android platform.
- First-class IDE support: Android Studio's code completion, refactoring, and debugging tools are built with Kotlin as a primary target language.

1.4 Why Are Emulators Used?

An Android emulator is a software-based virtual Android device that runs on a development computer. Emulators allow students to:

- Test an app on multiple device sizes, Android versions, and hardware configurations without purchasing physical devices.
- Develop and test even if no physical Android phone is available.
- Use built-in debugging tools such as Logcat, network throttling, and location simulation.
- Quickly reset to a clean device state between test runs.

Emulators trade a small amount of realism (true touch response, camera hardware, battery behaviour) for convenience and repeatability, which is why they are the standard day-to-day development tool and the primary test target for this lab.

1.5 Purpose and Learning Outcomes

By the end of this practical, you will be able to:

1. Install and configure Android Studio and the Android SDK on your own machine.
2. Diagnose and resolve common Android Studio and SDK setup issues.
3. Create and launch an Android Virtual Device (AVD).
4. Navigate the main panels of the Android Studio IDE.
5. Create a new Android project using Kotlin and understand its folder structure.
6. Write, build, and run a simple Kotlin Android application.

This lab is a one-time environment setup. Once completed, your machine will be ready for every subsequent Android development practical in this course.

1.6 Visual Overview

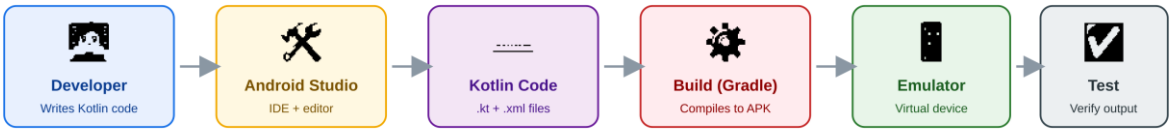


Figure 1.1 — The Android application development workflow: Developer → Android Studio → Kotlin Code → Build → Emulator → Test.

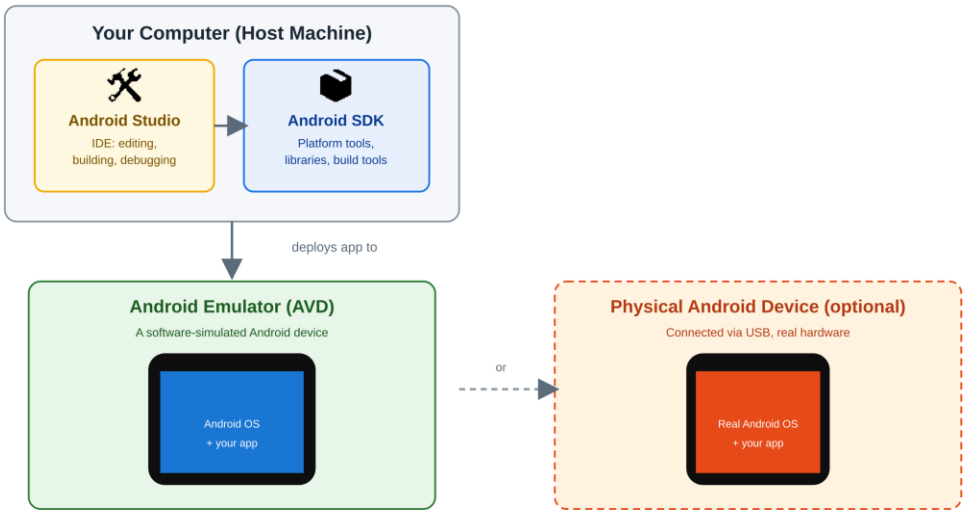


Figure 1.2 — How Android Studio, the Android SDK, and the Emulator (or a physical device) interact.

✓ Checkpoint — confirm before continuing

- ☑ You can describe, in your own words, the role of Android Studio versus the Android SDK.
- ☑ You can explain why an emulator is useful even when a physical phone is available.
- ☑ You understand the overall path from writing Kotlin code to seeing it run on a device.

Part 2 — Installing Android Studio

This section installs Android Studio itself, which bundles the Android SDK installer, platform tools, and emulator components used throughout the rest of this lab.

2.1 Installation Process Overview

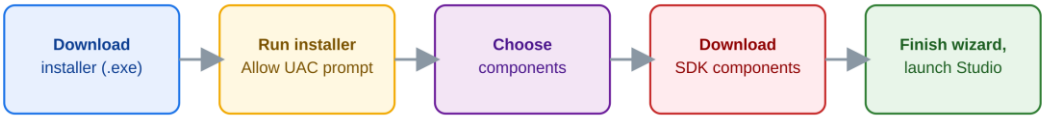


Figure 2.1 — The Android Studio installation process, start to finish.

2.2 Download Android Studio

1. Open a web browser and navigate to the official Android Studio download page: https://developer.android.com/studio
2. The page automatically detects your operating system and recommends the correct installer. Confirm that it has detected Windows (64-bit).
3. Click the green Download Android Studio button.
4. Read and accept the Android Software Development Kit License Agreement when prompted, then the download will begin (the installer is approximately 1–1.2 GB).

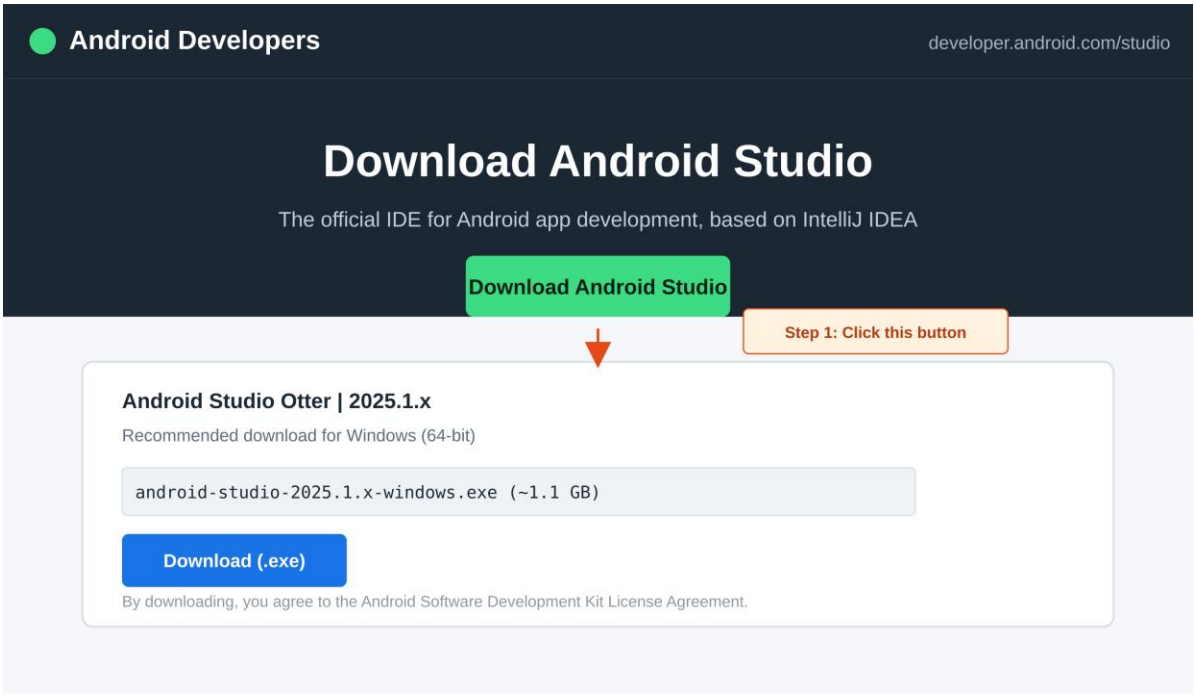


Figure 2.2 — Android Studio download page (illustrative layout; actual page styling may vary by release).

> **Tip**
> If your browser does not auto-detect Windows, scroll down on the download page to the "Platform" table and manually select the Windows (64-bit) .exe installer.

2.3 Install Android Studio

1. Locate the downloaded file (e.g., android-studio-2025.1.x-windows.exe) in your Downloads folder and double-click it to launch the installer.
2. If Windows shows a User Account Control prompt, click Yes to allow the installer to run.
3. On the Welcome screen, click Next.

2.3.1 Select Installation Components

The Choose Components screen lets you select which pieces to install:

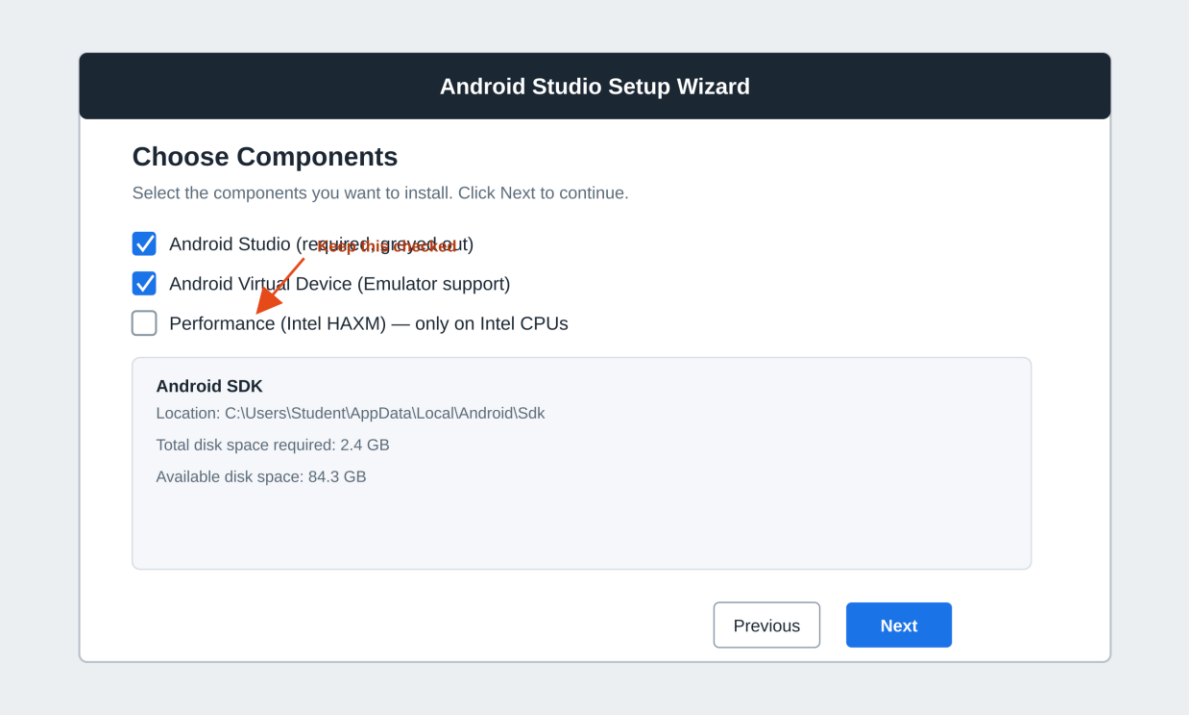


Figure 2.3 — Setup Wizard: Choose Components screen (illustrative).

- Android Studio — required; this checkbox is locked on.
- Android Virtual Device — keep this checked. It installs emulator support, needed for Part 4 of this lab.
- Performance (Intel HAXM) — only shown on Intel-based CPUs; leave at the default setting unless your instructor advises otherwise.

Click Next to continue.

2.3.2 Configure Android SDK Location

1. Accept the default installation path for Android Studio itself unless your system drive is low on space.
2. Note the Android SDK location shown on this screen (commonly C:\Users\<YourName>\AppData\Local\Android\Sdk). This is where the SDK, platform tools, and emulator system images will be stored.

3. Click Next, then click Install on the confirmation screen.

2.3.3 Complete the Setup Wizard

The installer will download and unpack several gigabytes of SDK components. This step depends on your internet connection and can take 10–30 minutes.

1. Wait for the progress bar to reach 100% across all components (Android SDK, SDK Platform-Tools, Emulator, etc.).
2. Click Next, then click Finish to close the installer.
3. Android Studio will launch automatically. On first launch you may be asked whether to import settings from a previous installation — choose Do not import settings if this is a fresh install.
4. Accept any remaining license agreements presented in the Android Studio Setup Wizard.

2.4 Verify the Installation

Once setup finishes, the Welcome to Android Studio screen should appear, offering options such as New Project, Open, and Get from VCS.

✓ Checkpoint — confirm before continuing

- ☑ Android Studio opens without error dialogs.
- ☑ Help → About shows a version number (e.g., Android Studio 2025.1.x).
- ☑ Help → SDK Manager shows at least one Android SDK Platform marked as installed.

If Android Studio fails to launch, see Part 9 (Troubleshooting) before proceeding.

Part 3 — Configuring the Android SDK

Although the installer in Part 2 sets up a working SDK by default, it is good practice for every Android developer to know how to inspect and manage SDK components directly. This section walks through the SDK Manager.

3.1 SDK Configuration Workflow

The general process for keeping an SDK installation healthy is: open the manager, check installed versus available components, install or update what is missing, then verify everything from the command line.

3.2 Open the SDK Manager

1. In Android Studio, go to File → Settings (Windows/Linux) or Android Studio → Settings (macOS).
2. In the left panel, expand Languages & Frameworks and select Android SDK.
3. Alternatively, click the SDK Manager icon directly in the toolbar, or use Tools → SDK Manager.

3.3 Install Required SDK Components

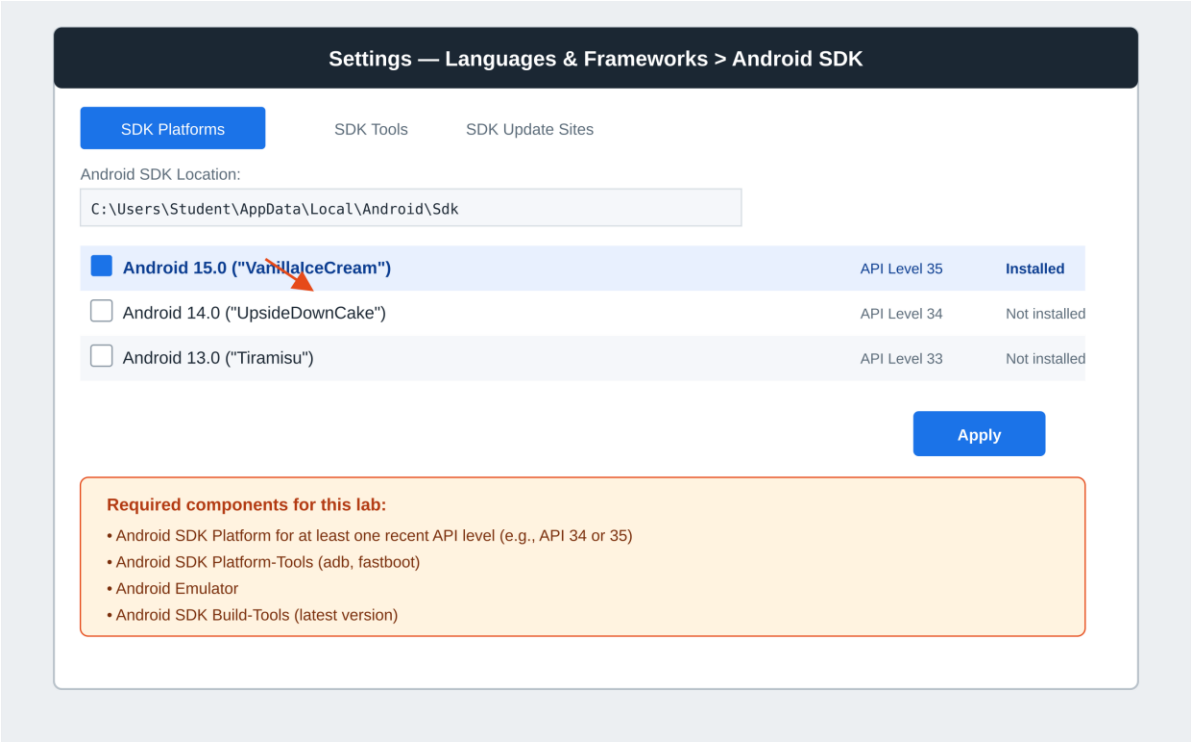


Figure 3.1 — The SDK Manager's SDK Platforms tab (illustrative).

1. On the SDK Platforms tab, check the box beside at least one recent Android version (API 34 or API 35 are good defaults for this lab).
2. Switch to the SDK Tools tab and ensure the following are checked: Android SDK Build-Tools, Android SDK Platform-Tools, Android Emulator, and Android SDK Command-line Tools.

3. Click Apply, review the list of components to be installed in the confirmation dialog, then click OK to begin the download.

3.4 Configure the SDK Location

The Android SDK Location field at the top of this settings page shows where all SDK files are stored on disk (for example C:\Users\Student\AppData\Local\Android\Sdk). You normally do not need to change this default, but it is useful to know this path if a tool ever asks for it.

3.5 Verify the Installation

1. Open a Command Prompt window (Start → type cmd → Enter).
2. Run the following command to confirm the platform tools are correctly installed and on your PATH:

Terminal command

```
adb --version
```

3. Confirm the output reports a version number rather than an error such as 'adb' is not recognized.

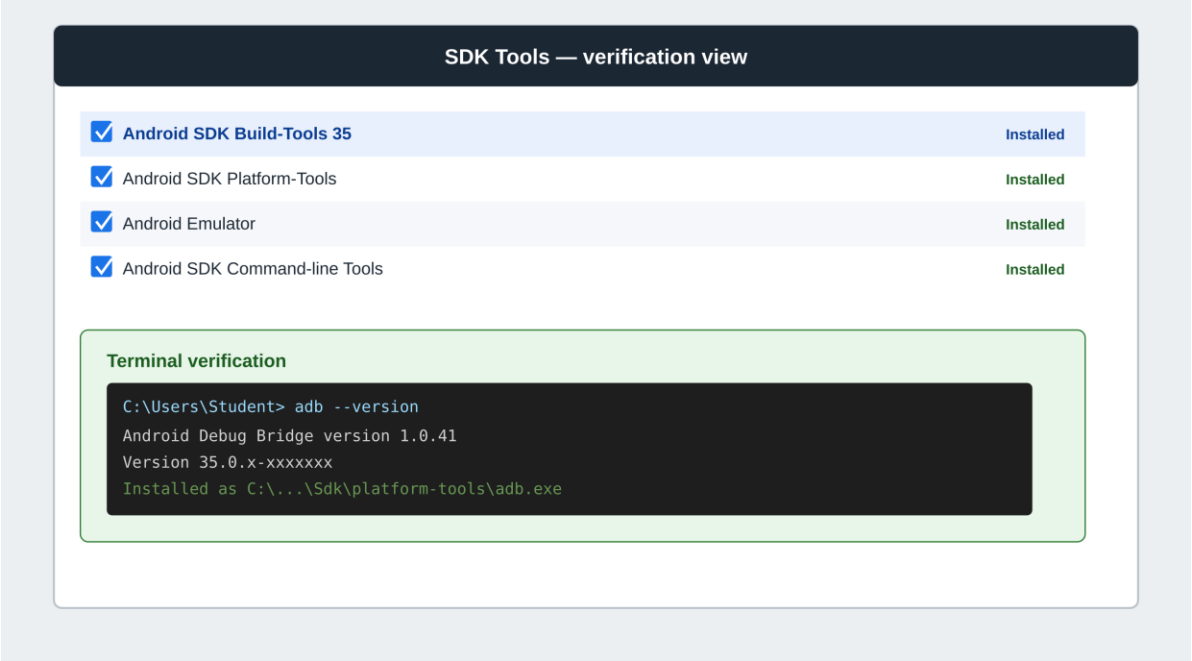


Figure 3.2 — Confirming the SDK installed correctly using adb --version (illustrative).

3.6 Update SDK Tools if Needed

Components occasionally have newer versions available. To update:

1. Reopen the SDK Manager (Tools → SDK Manager).
2. Check Show Package Details near the bottom-right of the panel to reveal individual version numbers.
3. Select a newer version of any component you wish to update, then click Apply.

Note

Updating SDK components requires an internet connection and can take several minutes depending on package size. It is safe to skip updates for this lab unless your instructor specifically asks for a newer version.

✓ Checkpoint — confirm before continuing

- ☑ The SDK Manager shows at least one Android SDK Platform as Installed.
- ☑ Android SDK Build-Tools, Platform-Tools, and Emulator all show as Installed under SDK Tools.
- ☑ Running adb --version in a terminal returns a version number, not an error.

Part 4 — Configuring the Android Emulator

An Android Virtual Device (AVD) is a configuration that defines a specific virtual phone or tablet, combining a hardware profile (screen size, RAM) with a system image (the actual Android OS version). This section creates one AVD to use as the test device for the rest of this lab.

4.1 Emulator Setup Workflow

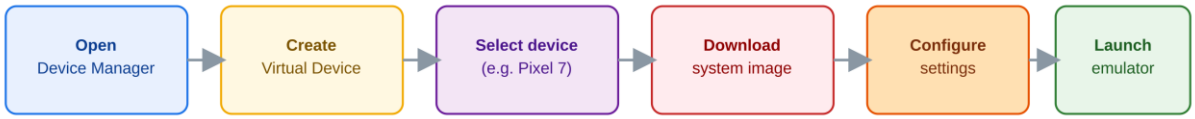


Figure 4.1 — Steps to create and launch a virtual Android device (AVD).

4.2 How the Emulator Relates to Your Computer

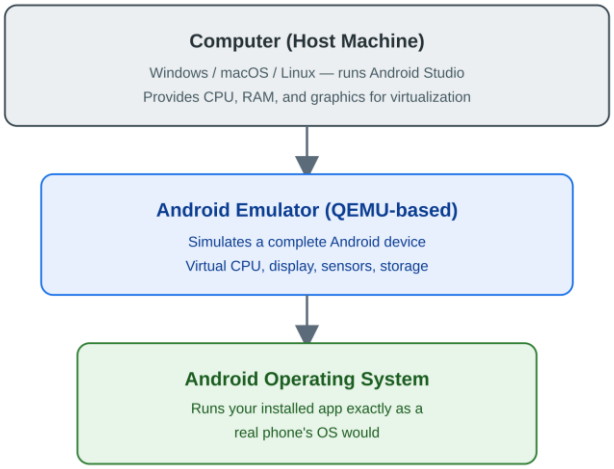


Figure 4.2 — Computer (host) → Emulator → Android Operating System.

Your physical computer provides the processing power; the Emulator software simulates an entire Android device on top of that hardware; and the simulated Android OS inside the emulator runs your app exactly as a real phone would.

4.3 Open the Device Manager

1. In Android Studio, open the Device Manager panel using one of the following: the icon in the top-right toolbar, or the menu path Tools → Device Manager.
2. If no virtual devices exist yet, the list will be empty except for a prompt to create one.

4.4 Create a Virtual Device

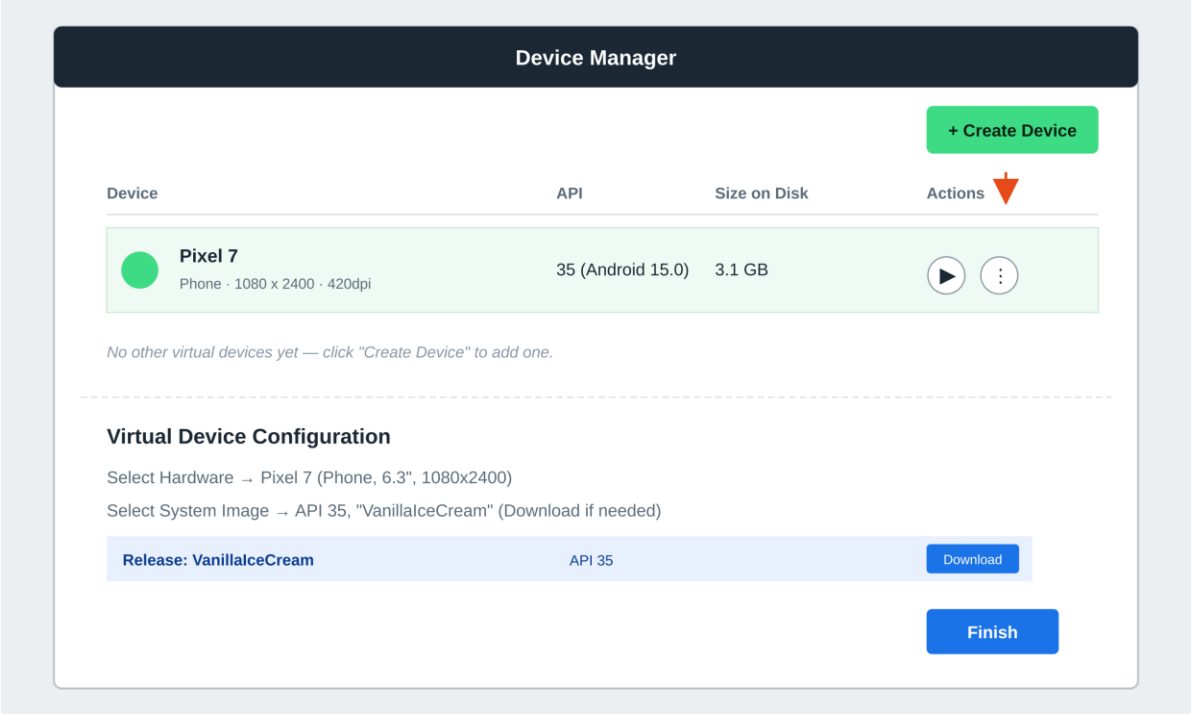


Figure 4.3 — Device Manager panel and the Virtual Device Configuration wizard (illustrative).

1. Click + Create Device near the top of the Device Manager panel.
2. In the Select Hardware step, choose a Phone category device. Pixel 7 or Pixel 9 are good defaults for this lab, offering a typical modern screen size.
3. Click Next.

4.4.1 Select and Download a System Image

1. On the System Image step, choose a recent Android release (the row marked Recommended is a safe default, typically the newest stable API level).
2. If the chosen image has not been downloaded yet, click the Download link next to it and wait for the download to complete (several hundred MB to ~1.5 GB depending on the image).
3. Once downloaded, select that system image and click Next.

4.4.2 Configure Emulator Settings

1. On the final configuration screen, you may optionally rename the AVD (e.g., Pixel_7_API_35).
2. Leave Startup orientation as Portrait and other advanced settings at their defaults for this lab.
3. Click Finish to create the AVD.

4.5 Launch the Emulator

1. In the Device Manager list, locate your newly created device.
2. Click the green Play (▶) icon in its row.
3. Wait for the emulator window to boot. The first boot of a freshly created AVD can take one to several minutes; subsequent boots are faster.

Tip

If the emulator window opens but stays black for longer than 2–3 minutes, see Part 9 (Troubleshooting — Emulator not starting) for common causes such as virtualization not being enabled in the BIOS.

✓ Checkpoint — confirm before continuing

- ☑ The emulator window opens and shows the Android home screen.
- ☑ The emulator responds to mouse clicks (acting as touch input).
- ☑ The device appears in Android Studio's device drop-down in the toolbar.

Part 5 — Exploring the Android Studio IDE

Before writing code, take a few minutes to learn the layout of the IDE. The diagram below labels the panels referenced throughout the rest of this manual.

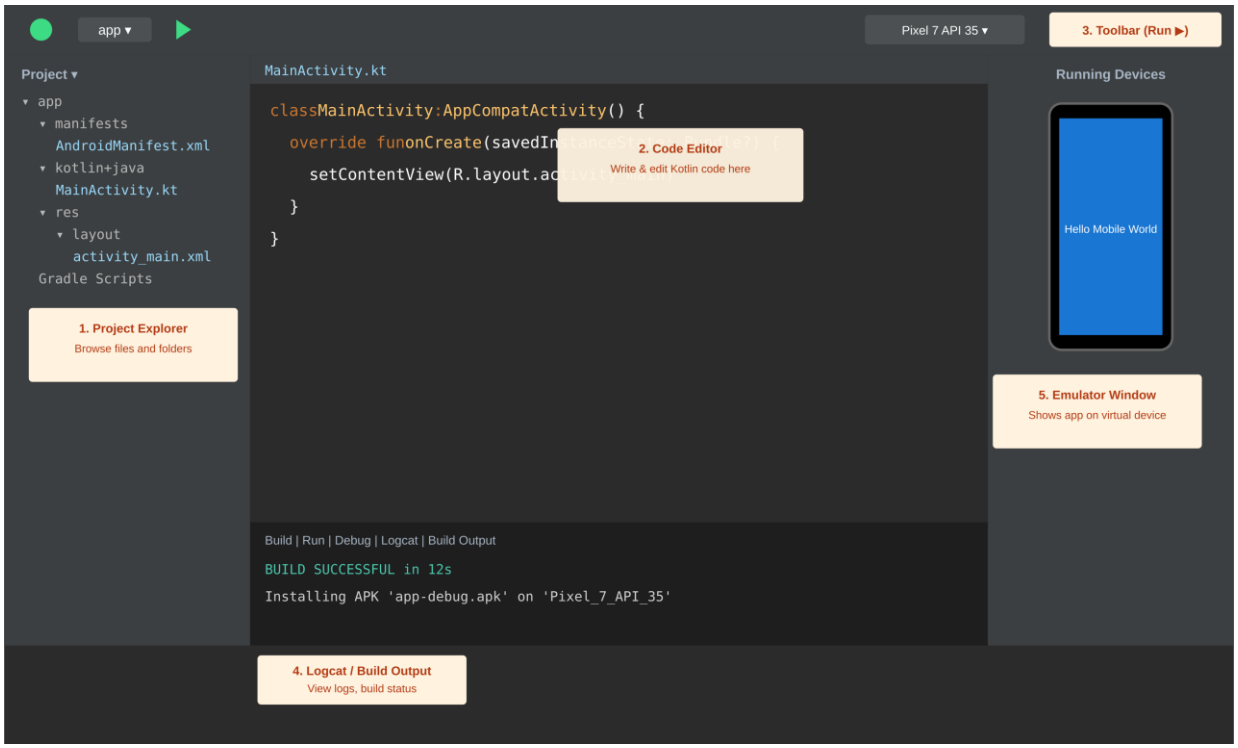


Figure 5.1 — Labelled overview of the Android Studio IDE layout for a Kotlin project (illustrative).

5.1 Project Explorer (label 1)

The vertical panel on the left lists every file and folder in your project in a tree view. Click the arrow beside a folder to expand or collapse it, and double-click a file to open it in the editor. The most important folders for this lab are kotlin+java (your code) and res/layout (your screen designs).

5.2 Code Editor (label 2)

The large central panel is where you read and write code. It provides syntax highlighting, auto-indentation, and code completion (press Ctrl+Space to manually trigger suggestions). A red squiggly underline indicates a syntax error; a yellow squiggly underline indicates a warning.

5.3 Toolbar (label 3)

The toolbar above the editor contains, from left to right: the project/module selector, the target device drop-down (used to choose which emulator or physical device to run on), and the Run (▶), Debug (bug icon), and Stop (■) buttons.

5.4 Device Manager

Covered in Part 4 — accessible from the toolbar icon or Tools → Device Manager, this panel lists and controls your virtual devices.

5.5 Run Button

The green triangle (▶) Run button compiles your code, packages it into an APK, installs that APK on the selected device, and launches the app, all without attaching a debugger.

5.6 Debug Button

The bug-shaped Debug button performs the same build-and-install process as Run, but additionally attaches a debugger, allowing you to set breakpoints and inspect variable values while the app executes.

5.7 Logcat (label 4, Logcat tab)

Logcat is a real-time log viewer showing system and app-level messages, including any print statements from your Kotlin code (via Log.d()) and runtime errors. Open it via the Logcat tab at the bottom of the IDE, or View → Tool Windows → Logcat. Use the search box at the top of the panel to filter for your app's package name or a specific keyword.

5.8 Build Output (label 4, Build tab)

The Build tab, next to Logcat, shows Gradle's build progress and any compilation errors. When a build fails, this is the first place to look — it reports the exact file and line number where a problem occurred.

5.9 Emulator Controls (label 5)

When a device is running, a slim control strip appears alongside the emulator window with buttons for Power, Volume, Rotate, and a Screenshot capture icon, along with an Extended controls (⋮) button that opens options for simulating GPS location, battery level, and more.

✓ Checkpoint — confirm before continuing

- ☑ You can locate and open the kotlin+java folder and the res/layout folder in the Project Explorer.
- ☑ You can open the Logcat panel and the Build panel.
- ☑ You can identify the Run, Debug, and Stop buttons in the toolbar.

Part 6 — Creating an Android Project Using Kotlin

6.1 Start a New Project

1. From the Android Studio Welcome screen, click New Project. (If a project is already open, use File → New → New Project… instead.)
2. In the project template gallery, select Empty Activity under the Phone and Tablet category.
3. Click Next.

6.2 Configure the Project

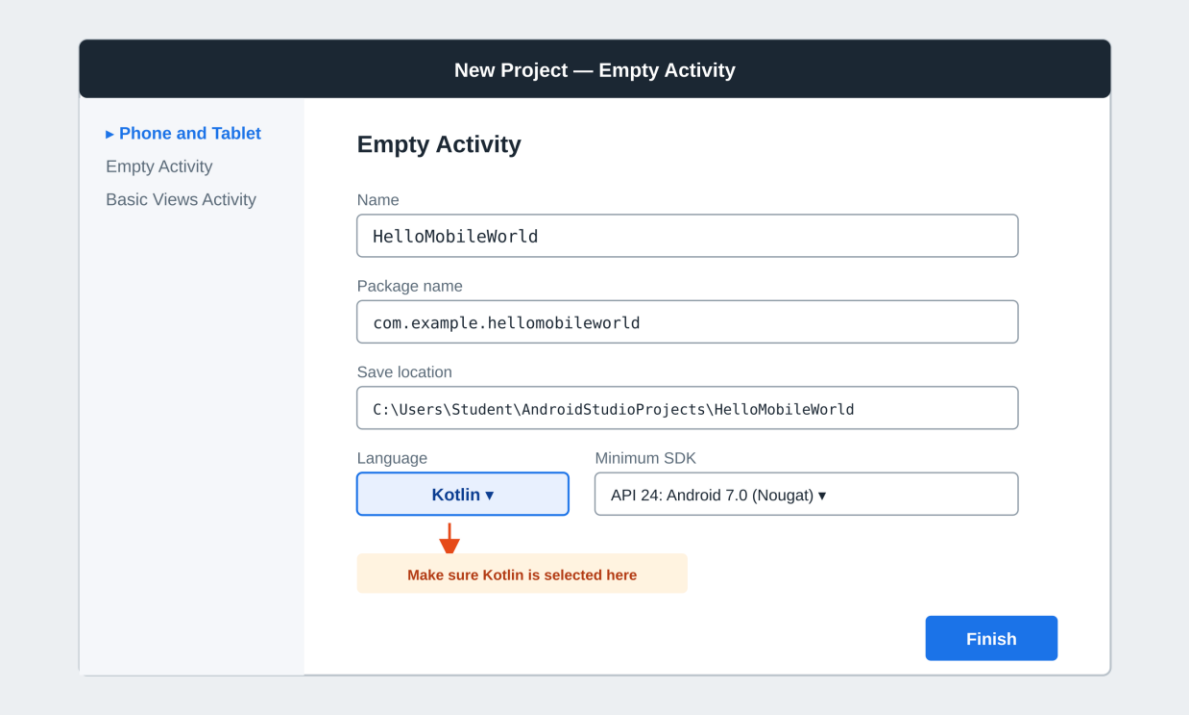


Figure 6.1 — New Project configuration screen with Kotlin selected as the language (illustrative).

1. Name: enter HelloMobileWorld.
2. Package name: accept the auto-generated value (e.g., com.example.hellomobileworld), or set your own following reverse-domain convention.
3. Save location: choose or confirm a folder such as C:\Users\<YourName>\AndroidStudioProjects\HelloMobileWorld.
4. Language: select Kotlin from the drop-down. This is the most important field to check for this lab.
5. Minimum SDK: choose API 24: Android 7.0 (Nougat) or higher, which covers the vast majority of active Android devices.
6. Click Finish. Android Studio will generate the project and run an initial Gradle sync — this may take one to two minutes on first creation.

6.3 Understanding the Generated Project Structure

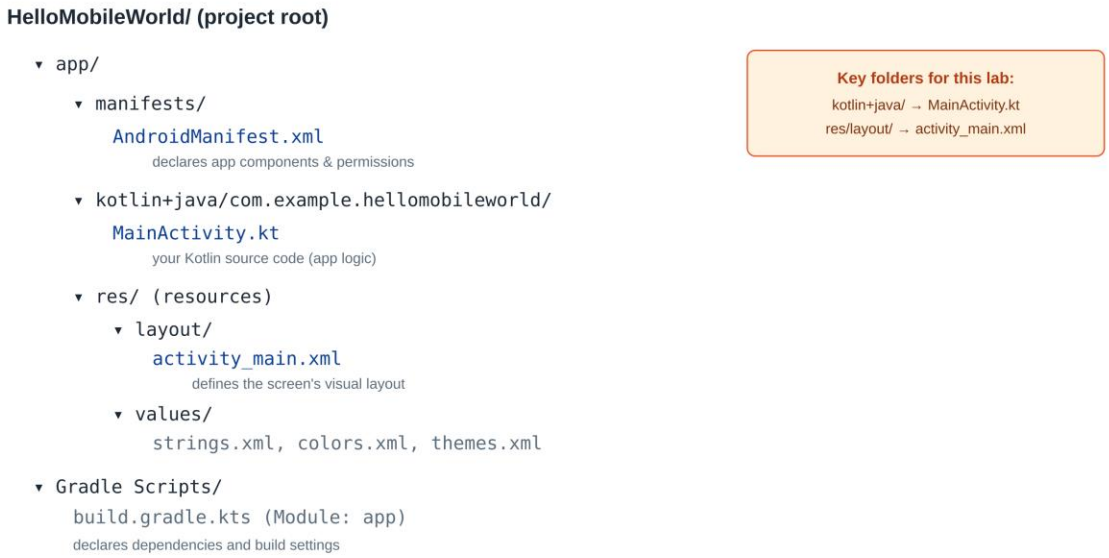


Figure 6.2 — Folder hierarchy of a newly created Kotlin Android project.

Folder / File	Purpose
app/manifests/AndroidManifest.xml	Declares the app's components (activities), required permissions, and the app's display name and icon.
app/kotlin+java/.../MainActivity.kt	Your Kotlin source code. Execution of the main screen begins in the onCreate() function here.
app/res/layout/activity_main.xml	Defines the visual layout of the main screen using XML — what widgets appear and how they are positioned.
app/res/values/strings.xml, colors.xml, themes.xml	Centralized resources for text strings, color values, and app-wide theming, kept separate from layout and code.
Gradle Scripts/build.gradle.kts (Module: app)	Declares the app's dependencies (external libraries), minimum/target SDK versions, and other build configuration.

Note

You will primarily edit two files in this lab: MainActivity.kt (the Kotlin code) and activity_main.xml (the screen layout). Everything else is generated scaffolding that Android Studio manages for you.

✓ Checkpoint — confirm before continuing

- ☑ The project named HelloMobileWorld appears in the Project Explorer.
- ☑ Gradle sync completes without errors (check the status bar at the bottom of the IDE).
- ☑ MainActivity.kt and activity_main.xml both exist and can be opened.

Part 7 — Developing the “Hello Mobile World” App

This app uses two files working together: an XML layout file that defines what appears on screen, and a Kotlin activity file that loads that layout when the app starts.

7.1 How the Files Work Together

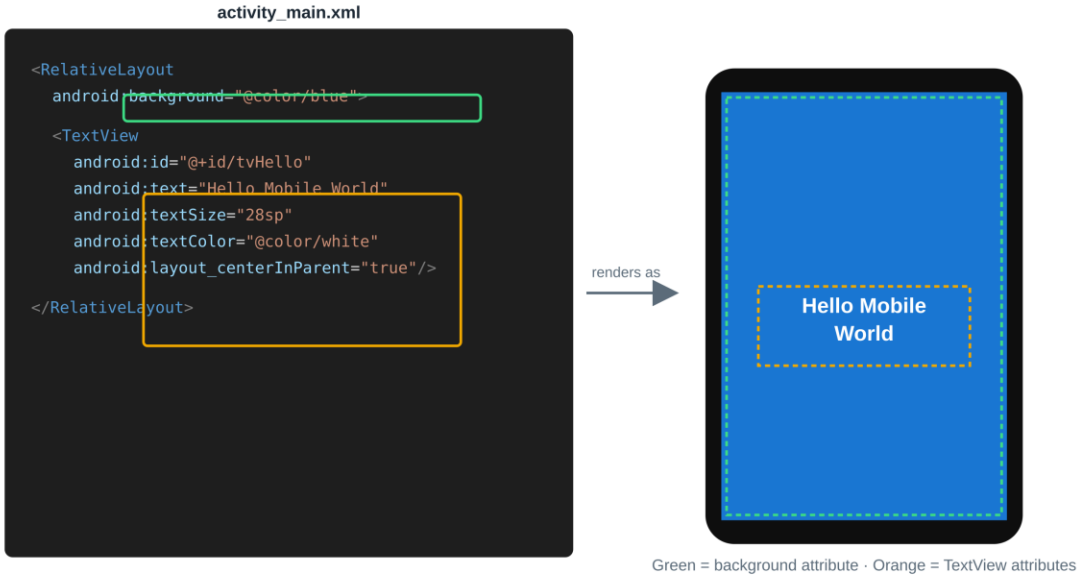


Figure 7.1 — How attributes in activity_main.xml map to what appears on the rendered screen.

7.2 Edit res/layout/activity_main.xml

1. In the Project Explorer, expand res → layout and double-click activity_main.xml to open it.
2. Click the Code tab in the top-right corner of the editor if the Design view opens by default — this lab edits the raw XML directly.
3. Select all existing content (Ctrl+A) and delete it.
4. Type or paste the complete layout shown below.

res/layout/activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#1976D2"
    tools:context=".MainActivity">

    <TextView
        android:id="@+id/tvHello"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_centerInParent="true"
        android:text="Hello Mobile World"
        android:textColor="#FFFFFF"
        android:textSize="28sp"
        android:textStyle="bold" />
```

```
</RelativeLayout>
```

7.3 Edit MainActivity.kt

1. In the Project Explorer, expand kotlin+java → com.example.hellomobileworld and double-click MainActivity.kt.
2. Select all existing content (Ctrl+A) and delete it.
3. Type or paste the complete code shown below, adjusting the package line at the top to match your own project's package name if it differs.

kotlin+java/.../MainActivity.kt

```
package com.example.hellomobileworld

import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }
}
```

7.4 Set the App Title

1. Expand res → values and open strings.xml.
2. Locate the line defining app_name and change its value to Hello Mobile World, as shown below.

res/values/strings.xml

```
<resources>
    <string name="app_name">Hello Mobile World</string>
</resources>
```

The app_name string is what AndroidManifest.xml references for the application's display name, shown in the app drawer, recent-apps switcher, and emulator title bar.

7.5 Line-by-Line Explanation

Code	Explanation
<RelativeLayout ...>	The root container for the screen. A RelativeLayout positions its children relative to the parent or to each other, which makes simple centering straightforward.
android:background="#1976D2"	Sets the background color of the entire screen to a blue shade, giving the app a deliberate visual theme.

<TextView android:id="@+id/tvHello" ...>	Declares a text-display widget and assigns it the ID tvHello so it could be referenced from Kotlin code if needed (not required for this simple version).
android:layout_centerInParent="true"	Positions the TextView in the exact horizontal and vertical center of its parent — this satisfies the “center alignment” requirement.
android:text="Hello Mobile World"	Sets the literal text displayed by the TextView, satisfying the core requirement of this lab.
android:textColor / android:textSize / android:textStyle	Apply basic styling: white text, 28sp size, and bold weight, satisfying the “basic styling” requirement.
package com.example.hellomobileworld	Declares which package this Kotlin file belongs to, matching the project's namespace.
class MainActivity : AppCompatActivity()	Defines the app's main screen as a class that inherits from AppCompatActivity, the standard base class for a single screen in a traditional (View-based) Android app.
override fun onCreate(savedInstanceState: Bundle?)	onCreate() is called by the Android system when the activity is first created — this is where setup code belongs.
super.onCreate(savedInstanceState)	Calls the parent class's setup logic, which must run before your own setup code.
setContentView(R.layout.activity_main)	Tells Android to inflate (render) the activity_main.xml layout and display it as this screen's content — this is the line that connects the Kotlin code to the XML layout.

✓ Checkpoint — confirm before continuing

- ☑ activity_main.xml and MainActivity.kt both have no red error markers in the editor's left margin.
- ☑ Every opening tag/bracket has a matching closing tag/bracket.
- ☑ Both files are saved (no unsaved-changes dot on either file tab).

Part 8 — Running and Testing the App

8.1 What Happens When You Click Run



Figure 8.1 — What happens behind the scenes when you click the Run button.

8.2 Select the Emulator as the Run Target

1. Ensure the AVD created in Part 4 is running (if not, launch it from the Device Manager).
2. In the Android Studio toolbar, click the device drop-down and confirm your emulator (e.g., Pixel 7 API 35) is selected.

8.3 Build and Run the Project

1. Click the green Run (▶) button in the toolbar, or press Shift+F10.
2. Android Studio invokes Gradle to compile the Kotlin code, package an Android package (APK), install it on the emulator, and launch it. The first run of a project is the slowest (often 30–90 seconds) because Gradle configures the build for the first time.
3. Watch the Build panel at the bottom of the IDE for progress messages; a successful build ends with BUILD SUCCESSFUL.

8.4 Observe the Output

Once the build finishes, the app launches automatically on the emulator and should match the following:

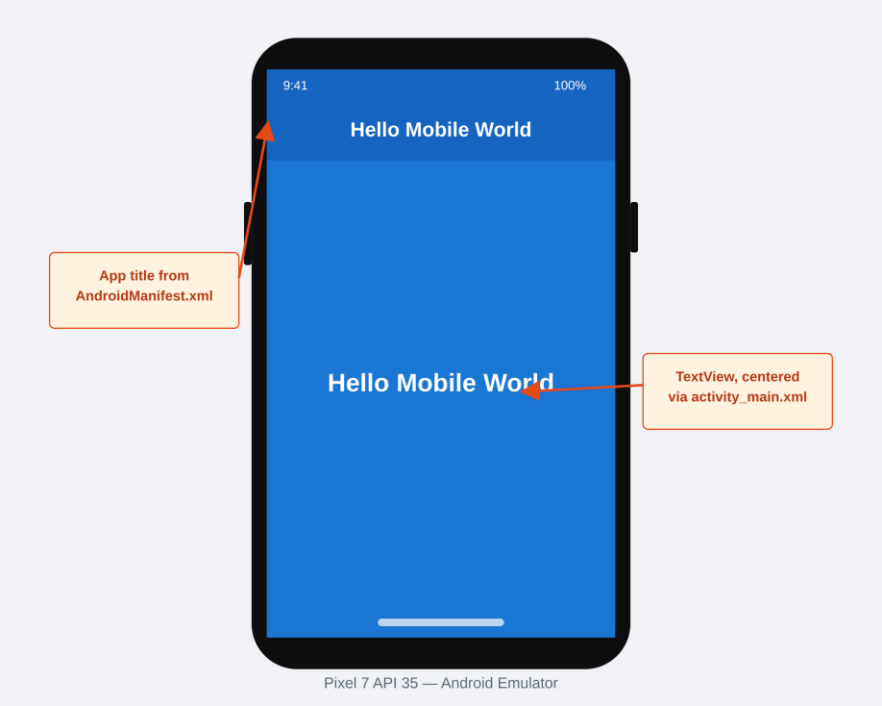


Figure 8.2 — Hello Mobile World running on the emulator (illustrative).

- A blue screen background, matching the color set in activity_main.xml.
- The message “Hello Mobile World” centered on screen, in bold white text.
- The app's title bar / task-switcher entry reading “Hello Mobile World,” matching the app_name string.

8.5 Demonstrate Rerun and App Update

A core part of Android development is the edit-build-run cycle. Practice it now:

1. With the app still running, return to activity_main.xml and change android:textSize from 28sp to 36sp (or change the background color hex code to a different value).
2. Save the file (Ctrl+S).
3. Click the Run (▶) button again. Android Studio rebuilds and reinstalls the updated APK on the same emulator.
4. Confirm the new text size or background color appears once the app relaunches.

Note
Unlike frameworks that support Hot Reload, a traditional Kotlin/XML Android app must be rebuilt and reinstalled to see changes. This typically still only takes a few seconds after the first build, since Gradle caches most of the work.

✓ Checkpoint — confirm before continuing

- ☑ The app builds and launches on the emulator without errors.
- ☑ The screen shows the blue background and the centered, styled “Hello Mobile World” text.
- ☑ A change to the layout (e.g., text size or background color) appears after rerunning the app.

Part 9 — Troubleshooting

The table below lists issues commonly encountered during this lab, organized using a consistent pattern: what you see, why it happens, and how to fix it.



Figure 9.1 — The Error → Cause → Fix pattern used throughout this section.

9.1 Android SDK Not Found

Symptom (Error)	Cause	Fix
Android Studio shows “SDK location not found” on startup	The SDK path was not configured, or was pointed at a folder that no longer exists.	Go to File → Settings → Languages & Frameworks → Android SDK and browse to the correct SDK folder, then click Apply.
Gradle error: SDK location not found. Define a valid SDK location with an ANDROID_HOME environment variable	The project's local.properties file is missing or has an invalid sdk.dir path.	Let Android Studio regenerate this file automatically by re-opening the project, or manually set sdk.dir in local.properties to your SDK path.

9.2 Emulator Not Starting

Symptom (Error)	Cause	Fix
Emulator window opens but stays black for several minutes	First boot of a new AVD is naturally slow.	Wait at least 3–5 minutes before assuming failure. If it still fails, wipe the device's data from the Device Manager's actions menu and relaunch.
Error mentioning HAXM, VT-x, or virtualization	Hardware virtualization is disabled in the BIOS/UEFI, or is blocked by Hyper-V/WSL on Windows.	Enable Intel VT-x / AMD-V in BIOS; consider disabling Hyper-V if it conflicts, or use a Google APIs (non-Play Store) system image, which is less resource-intensive.
Emulator is extremely slow	Insufficient RAM allocated to the AVD, or software rendering being used instead of hardware acceleration.	Increase the AVD's allocated RAM in its settings and confirm “Hardware - GLES 2.0” is selected as the graphics setting.

9.3 Gradle Sync Failures

Symptom (Error)	Cause	Fix
Gradle sync failed: could not resolve dependencies	No internet connection during the first sync, or a firewall is blocking Maven/Google repositories.	Confirm internet connectivity, then click Try Again in the sync notification, or File → Sync Project with Gradle Files.
Build fails after an Android Studio update with a version mismatch	The project's Gradle wrapper version is incompatible with the newly updated Android Gradle Plugin.	Use Android Studio's automatic upgrade prompt (Upgrade Gradle), or run File → Invalidate Caches and restart.
Build hangs at “Configure project :app” for a long time	Antivirus software scanning the project's build/ and .gradle/ folders on every file write.	Add the project folder and the .gradle cache folder to your antivirus's exclusion list.

9.4 Kotlin Compilation Errors

Symptom (Error)	Cause	Fix
Unresolved reference: R	The project has not been synced/built since the layout file was created or edited, so the auto-generated R class is out of date.	Click Build → Make Project (Ctrl+F9), or File → Sync Project with Gradle Files.
Type mismatch: inferred type is X but Y was expected	A value of the wrong type was assigned to a variable or passed to a function — Kotlin's null-safety and strict typing catch this at compile time.	Read the exact line number in the error message and correct the variable's declared type or the value being assigned.
Expecting a top level declaration / mismatched braces	A missing or extra curly brace { } somewhere above the reported line.	Use the editor's bracket-matching highlight (click next to a brace) to trace mismatched pairs, working outward from the reported error line.

9.5 Device Connection Issues

Symptom (Error)	Cause	Fix
Device drop-down in the toolbar is empty	The emulator has not finished booting, or no physical device is connected.	Wait for the emulator to fully boot to the home screen, or connect a physical device via USB.
Physical device not detected	USB debugging is not enabled, or the device has not approved the computer's debugging request.	Enable Developer Options (Settings → About phone → tap Build number 7 times), enable USB debugging, and accept the “Allow USB debugging” prompt on the phone.

9.6 Installation / Build Configuration Issues

Symptom (Error)	Cause	Fix
INSTALL_FAILED_INSUFFICIENT_STORAGE	The emulator's virtual storage is full from repeated installs.	Wipe the AVD's data from the Device Manager's actions menu, or increase the AVD's internal storage in its settings.
Manifest merger failed	Two dependencies (or the manifest and a dependency) declare conflicting attributes.	Read the specific conflicting attribute named in the error and add a tools:replace or tools:node override in AndroidManifest.xml as directed by the message.

General first step

Whenever something unexpected happens, check the Build panel and Logcat first — both report the exact file, line number, or stack trace behind almost every issue in this lab.

Part 10 — Reflection Questions and Extensions

10.1 Discussion Questions

1. Android Studio, the Android SDK, and the Android Emulator are three separate components that work together. Explain the distinct role each one plays, and describe what would happen if only one of the three were missing.
2. Kotlin's null-safety system distinguishes nullable types (e.g., String?) from non-nullable types (e.g., String) at compile time. Explain why this matters for app reliability compared to a language that does not enforce this distinction, and describe a situation in this lab's code where a nullable type appears.
3. This lab uses an XML layout file together with a separate Kotlin activity file, rather than building the UI entirely in code. Discuss one advantage of separating layout from logic, and explain how setContentView(R.layout.activity_main) connects the two files at runtime.
4. Emulators are convenient but cannot perfectly replicate every aspect of a physical device. Discuss at least two categories of bugs that might only appear on a physical device and not on an emulator.
5. During Part 9, the Error → Cause → Fix pattern was used to organize troubleshooting. Pick one error from that section and explain, in your own words, why understanding the cause (not just memorizing the fix) makes you a more effective developer for future, unfamiliar errors.

10.2 Practical Tasks / Extensions

1. Change the text color of the “Hello Mobile World” message and the background color of the screen to two different colors of your choice, then rebuild and rerun the app to confirm the changes.
2. Add a Button below the existing TextView using a LinearLayout (vertical orientation) instead of a RelativeLayout, and use Kotlin code in onCreate() to display a Toast message when the button is tapped.
3. Add a second screen (a new Empty Activity) and use an Intent in Kotlin to navigate from the first screen to the second when a button is pressed, demonstrating basic multi-screen navigation.

Appendix A — Quick Reference

Action	Menu Path / Shortcut
Open SDK Manager	Tools → SDK Manager
Open Device Manager	Tools → Device Manager
Run the app	Run ▶ button, or Shift+F10
Debug the app	Debug icon, or Shift+F9
Open Logcat	View → Tool Windows → Logcat
Build / rebuild without running	Build → Make Project, or Ctrl+F9
Sync Gradle files	File → Sync Project with Gradle Files
Clear cached state after odd errors	File → Invalidate Caches…
Check Android SDK location	File → Settings → Languages & Frameworks → Android SDK
Verify platform tools from terminal	adb --version

Appendix B — Submission Checklist

- Screenshot of adb --version confirming a working SDK installation.
- Screenshot of the running emulator displaying the completed Hello Mobile World app.
- Final source code for activity_main.xml and MainActivity.kt (as files or pasted text, per your instructor's submission format).
- Written answers to the five discussion questions in Section 10.1.
- Evidence (screenshot or short description) of completing at least one practical extension from Section 10.2.